Stateless Load Balancer

stateless load balancer

A **Stateless Load Balancer** is a load balancer that **does not store any information about previous client requests**. Each incoming request is treated **independently**, without remembering what happened before.

1️⃣ Simple Definition

A **stateless load balancer** distributes requests to backend servers **without maintaining session information** about clients.

So every request is handled **as a completely new request**.

2️⃣ Example

Suppose you have 3 servers:

- Server A
- Server B
- Server C

A stateless load balancer may distribute requests like this:

Request	Server
Request 1	Server A
Request 2	Server B
Request 3	Server C
Request 4	Server A

The load balancer **doesn't care which server handled previous requests from the same user**.

3️⃣ How it Works

It uses algorithms like:

- **Round Robin**

- **Least Connections**
- **Random**
- **Weighted Round Robin**

These algorithms decide **which server should handle the request**.

4️⃣ Example in Microservices

In a **Spring Boot microservices architecture**, tools like:

- Spring Cloud LoadBalancer
- Netflix Ribbon
- NGINX

can behave **statelessly**.

Each request is routed to any available service instance.

Example:

```
User → Load Balancer → Order Service Instance 1
User → Load Balancer → Order Service Instance 2
User → Load Balancer → Order Service Instance 3
```

The load balancer **does not remember the user**.

5️⃣ Why Stateless is Preferred in Microservices

Stateless systems are easier to scale.

Advantages:

✅ Easy horizontal scaling
✅ No session management required
✅ Fault tolerant
✅ Any instance can handle any request

6️⃣ Problem with Stateless Load Balancer

If your application stores **session data in the server memory**, then stateless load balancing causes problems.

Example:

```
Login Request → Server A
Next Request → Server B
```

Server B **does not know the session**, because it was stored in Server A.

7️⃣ Solution

Store session data in **shared storage**, such as:

- Redis
- Memcached
- Database

Then **any server can access the session**.

8️⃣ Stateless vs Stateful Load Balancer

Feature	Stateless	Stateful
Session storage	No	Yes
Request dependency	Independent	Depends on previous requests
Scalability	High	Lower
Example	API gateway, microservices	Sticky sessions

✅ Interview one-line answer

> A **stateless load balancer distributes requests to servers without storing any client session or previous request information. Each request is treated independently.**

If you want, I can also explain a **very important microservices interview concept:**

Stateless Microservice vs Stateful Microservice (with Netflix architecture example) — this is asked very frequently in **Spring Boot interviews.** 🚀

stateless vs stateful

Stateless vs Stateful

These terms describe **whether a system remembers previous interactions with a client**.

1️⃣ Stateless

A **stateless system does not store any client session or previous request data**.
Each request is treated **independently**.

Example

Suppose a user sends two requests:

```
Request 1 → Login
Request 2 → Get Profile
```

In a stateless system:

- The server **does not remember the login request**.
- Each request must contain **all required information**.

Example using token:

```
Request 1 → Login → server returns JWT
Request 2 → Get Profile → client sends JWT again
```

Technologies often used:

- REST APIs
- JWT
- Redis (for shared session storage)

Advantages

✅ Easy to scale horizontally
✅ No dependency between requests
✅ Load balancer can send request to **any server**

Example architecture

```
Client
   |
Load Balancer
   |
------------------------
| Server1 | Server2 | Server3 |
------------------------
```

Any request can go to **any server**.

2️⃣ Stateful

A **stateful system remembers previous interactions with the client**.

The server stores **session data**.

Example

```
Request 1 → Login → Session created in Server A
Request 2 → Get Profile → Must go to Server A
```

If the request goes to **Server B**, it won't know the session.

So **sticky sessions** are required.

Example

```
Client
   |
Load Balancer (Sticky Session)
   |
Server A (Session stored here)
```

3️⃣ Key Differences

Feature	Stateless	Stateful
Server memory of client	❌ No	✅ Yes
Session stored	Client / shared storage	Server
Scalability	Very high	Limited
Load balancing	Easy	Needs sticky sessions
Microservices usage	Preferred	Rare

4️⃣ Real-World Examples

Stateless systems

- REST APIs
- JWT authentication
- Microservices architectures

Stateful systems

- Traditional web apps with **HTTP sessions**
- Online games maintaining user state
- Database connections

5️⃣ Microservices Best Practice

Modern architectures (like those using Spring Boot) prefer **stateless services**.

Reason:

- Easy scaling
- Fault tolerance
- Cloud friendly

✅ Interview one-line answer

> **Stateless systems do not store client state between requests, while stateful systems maintain session information about the client.**

If you want, I can also explain **one tricky interview question related to this:**

"If microservices must be stateless, then how does login/session work?"

This confuses many developers in interviews.

yes

Good question 👍 — this is a **very common interview confusion**.

If **microservices should be stateless**, then **how does login or session management work?**

The answer is: **the state is not stored in the service memory**.

Instead, we use **tokens or external storage**.

1️⃣ Problem if we store session in server (Stateful)

Example:

```
User → Login → Server A creates session
```

Session stored in **Server A memory**.

Next request:

```
User → Load Balancer → Server B
```

Server B **doesn't know the session**, so the request fails.

This breaks **scalability**.

2️⃣ Solution 1 — Token-Based Authentication (Most Common)

Modern systems use **tokens** like
JSON Web Token.

Flow

```
1 User → Login → Auth Service
2 Auth Service → generates JWT
3 JWT sent to client
4 Client sends JWT in every request
```

Example request:

```
GET /orders
Authorization: Bearer JWT_TOKEN
```

Now **any service instance can verify the token**.

```
Client
   |
Load Balancer
   |
----------------------
| Service1 | Service2 |
----------------------
```

Since the token contains user info, the service **does not need stored session**.

So the system remains **stateless**.

3️⃣ Solution 2 — Shared Session Store

Instead of storing sessions in server memory, store them in a **central store** like:

- Redis
- Database

Flow

```
Login → Session stored in Redis
```

Then:

```
Request → Server1 → check Redis
Request → Server2 → check Redis
```

Any server can read the session.

4️⃣ Architecture in Microservices

Typical architecture using Spring Boot:

```
Client
   |
API Gateway
   |
Auth Service (creates JWT)
   |
-----------------------------
| Order Service | User Service |
-----------------------------
```

Client sends **JWT with every request**, so services remain **stateless**.

5️⃣ Interview Ready Answer

If interviewer asks:

“Microservices should be stateless. How do you manage user sessions?”

You can answer:

> In microservices, we avoid storing sessions in service memory. Instead, we use **token-based authentication like JWT** or **shared session storage such as Redis**, so any service instance can handle the request without maintaining state.

✅ **Pro tip for interviews**

They often ask a follow-up:

"Why is stateless architecture better for microservices?"

Answer in one line:

> Because it allows **horizontal scaling, fault tolerance, and easier load balancing**.

If you want, I can also explain **a very important interview topic related to this:**

Sticky Sessions vs Stateless Load Balancing (with diagrams) — this is often asked in **Spring Boot / Microservices interviews.** 🚀